Towards partial fulfillment for Undergraduate Degree Level Programme

Bachelor of Technology in Computer Science and Engineering

CS402 - INDUSTRIAL TRAINING

A Project Report on: Wells Fargo Internship

Prepared by:

Admission Number	Student Name
U22CS123	Vatsal Koisa

Semester: VIII

Year: 2025–2026

Supervised by: Dr. Naveen Kumar

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

S. V. NATIONAL INSTITUTE OF TECHNOLOGY (NIT)

SURAT - 395007 (GUJARAT, INDIA)

Student Declaration

This is to certify that the work described in this project report entitled **Wells Fargo Internship** has been actually carried out and implemented by me. Neither the source code therein, nor the content of the project report have been copied or downloaded from any other source. I understand that my result grade would be revoked if later it is found to be so.

Sr. No.	**Admission No.**	**Student Name**	**Signature of the Student**
1	U22CS123	Vatsal Koisa	

Certificate

This is to certify that the project report entitled **Wells Fargo Internship** is prepared and presented by:

Sr. No.	Admission No.	Student Name
1	U22CS123	Vatsal Koisa

student of B. Tech. Computer Science and Engineering and the work is satisfactory.

Signatures:

Supervisor **Jury** **Head of Department**

Abstract

This report presents the technical outcomes of my industrial internship at Wells Fargo, focused on modernization of decisioning systems, cost optimization of decision infrastructure, and real-time fraud data centralization. The first workstream addressed decommissioning and modernization of a retail credit decisioning platform by consolidating repositories, automating rule compilation to `.adb` artifacts, publishing artifacts to Artifactory, and introducing artifact versioning for traceability and controlled rollback. This significantly reduced manual effort and improved release reliability for business-driven rule changes.

The second workstream explored an in-house local decision stand-in server as a strategic alternative to expensive third-party decision engines. A prototype architecture using Java Spring Boot and React with TypeScript was designed to support backup-first deployment, operational continuity during vendor outages, and gradual migration based on business confidence.

The third workstream proposed and implemented the core design for a centralized fraud transaction repository using Kafka-based event streaming. Incoming fraud-check requests are published by producers, consumed by listeners, and persisted in a central repository with subsequent status updates after investigation. This architecture improves cross-team visibility and helps prevent inconsistent approval decisions during active identity theft scenarios.

Overall, the internship demonstrates how event-driven integration, CI/CD automation, and architecture simplification can jointly improve speed, reliability, and governance in financial systems. The report also documents limitations, rollout considerations, and key learnings, including improved productivity through disciplined use of GitHub Copilot for code generation, documentation drafting, and iterative technical refinement.

Table of Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Project Objectives 2

1.3 My Contribution 2

1.4 Outline of the Report 3

2 Related Work **4**

2.1 Credit Decisioning Lifecycle in Enterprises 4

2.2 Stand-in and Fallback Decision Services 4

2.3 Event-Driven Fraud Data Synchronization 5

2.4 Gap Addressed by This Internship Work 5

3 Methodology **6**

3.1 Workstream Decomposition 6

3.2 Requirement Gathering and Validation 6

3.3 Method for Workstream 1: Decisioning Automation 7

3.4 Method for Workstream 2: Stand-in Server Strategy 7

3.5 Method for Workstream 3: Fraud Centralization Pipeline 7

3.6 Evaluation Framework 8

3.7 Assumptions and Constraints 8

4 Implementation **9**

4.1 Workstream 1: Credit Decisioning Platform Modernization 9
4.1.1 Initial State 9
4.1.2 Repository Consolidation 9
4.1.3 Automated Rule Build Pipeline 9
4.1.4 Artifact Publishing and Versioning 10
4.2 Workstream 2: In-house Decision Stand-in Server 10
4.2.1 Architecture Selection 10
4.2.2 Operational Role 10
4.2.3 Migration Strategy 10
4.3 Workstream 3: Fraud Central Repository with Kafka 11
4.3.1 Problem-Driven Design 11
4.3.2 Event Flow Implementation 11
4.3.3 Core Design Considerations 11
4.4 Technology Stack Summary 11
4.5 Implementation Limitations 11

5 Results and Discussion 13
5.1 Results of Decisioning Modernization 13
5.2 Results of Stand-in Server Initiative 13
5.3 Results of Fraud Centralization Pipeline 14
5.4 Measurable Outcomes Summary 14
5.5 Limitations and Risks 15
5.6 Key Learnings from Discussion 15

6 Conclusions and Future Works 16
6.1 Conclusions 16
6.2 Future Works 16
6.3 Personal Learning Outcomes 17

6.3.1 Technical and Professional Learning 17
6.3.2 Learning to Use GitHub Copilot Better 17

Bibliography 18

List of Tables

4.1 Technology stack used across internship workstreams. 12

5.1 Measurable outcomes framework across internship initiatives. 14

1
Introduction

Financial institutions operate in highly regulated, high-volume environments where decision latency, operational resilience, and data consistency directly influence customer trust and business performance. During my internship at Wells Fargo, I worked on three high-impact engineering initiatives that addressed critical concerns in retail credit decisioning and fraud handling. These initiatives were selected because they sit at the intersection of software architecture, business continuity, and compliance-oriented governance.
The first initiative focused on modernization of a credit decisioning platform where rule updates were dependent on manual compilation and fragmented repository management. The second initiative explored an internal decision stand-in server to reduce third-party dependence and recurring licensing costs while improving service continuity. The third initiative addressed a fraud operations gap by designing a near real-time centralized repository for blocked and under-investigation transactions using Kafka and Spring Boot. Taken together, these projects represent a practical transformation journey: from manually operated, distributed processes to automated, event-driven, and traceable enterprise workflows.

1.1 Motivation

The internship problem statements were motivated by real operational pain points observed in production-grade environments:

- **Manual release bottlenecks:** business rule changes in credit decisioning required manual compilation and packaging, increasing delay and error probability.
- **High external dependency costs:** third-party cloud and local licensed decision engines incurred substantial recurring expenses.
- **Fraud data visibility gap:** absence of a centralized fraud repository created the risk of inconsistent decisions across internal teams.

In banking systems, these challenges are not isolated engineering inconveniences. They can impact policy rollout speed, fraud response time, and compliance readiness. Therefore, each initiative was approached with both technical correctness and business utility in mind.

1.2 Project Objectives

The end-sem internship work was structured around the following objectives:

- Automate the rule compilation and artifact lifecycle for credit decisioning.
- Reduce platform complexity through repository consolidation and standardized build flow.
- Design an in-house stand-in decisioning server architecture for continuity and long-term cost optimization.
- Implement an event-driven data pipeline for near real-time fraud transaction centralization.
- Improve governance through versioning, traceability, and well-defined integration patterns.

1.3 My Contribution

The specific contributions I delivered during the internship are summarized below:

1. **Credit decisioning modernization:**
   - Contributed to consolidation of utility repositories into a central platform.
   - Implemented CI-triggered automation for rule compilation into `.adb` files.
   - Added automated publishing to Artifactory with versioned artifacts.
2. **Decision stand-in server initiative:**
   - Participated in architecture and prototype planning for a local stand-in service.
   - Defined stack using Java Spring Boot (backend) and React + TypeScript (frontend).
   - Proposed backup-first deployment strategy to reduce migration risk.
3. **Fraud centralization pipeline:**

- Designed Kafka producer-consumer flow for incoming fraud-check events.
- Integrated listener-driven persistence into a centralized repository.
- Structured post-investigation status update handling for lifecycle completeness.

1.4 Outline of the Report

The remainder of this report is organized as follows:

- **Chapter 2: Related Work** discusses industry and architectural context relevant to decisioning automation, stand-in services, and event-driven fraud systems.
- **Chapter 3: Methodology** presents the engineering approach, workstream decomposition, and evaluation criteria used during execution.
- **Chapter 4: Implementation** provides the technical details of each initiative, including data flow and integration decisions.
- **Chapter 5: Results and Discussion** analyzes business impact, technical outcomes, limitations, and operational considerations.
- **Chapter 6: Conclusions and Future Works** concludes the report with major learnings, future roadmap, and reflections from the internship.

2

Related Work

This chapter positions the internship work against common enterprise practices in decisioning and fraud systems.

2.1 Credit Decisioning Lifecycle in Enterprises

In many financial organizations, decisioning rules are authored by business policy teams and translated into executable rule artifacts by engineering teams. Legacy operational models often depend on manual packaging and controlled deployment windows. While these processes provide governance checkpoints, they become bottlenecks when rule updates are frequent.

Modern practice emphasizes CI/CD-supported rule compilation, artifact repositories, and strict versioning to ensure reproducibility and rollback capability. The modernization work in this report aligns with this transition by automating conversion of rule updates into deployable `.adb` artifacts and publishing them through an auditable artifact pipeline.

2.2 Stand-in and Fallback Decision Services

Third-party decision engines are widely used because they offer mature capabilities and quick onboarding. However, dependence on external services introduces recurring cost and operational risk. A common resilience pattern in distributed systems is to maintain an internal fallback or stand-in service that can temporarily absorb traffic during external outages.

Existing architectural strategies include:

- Active-passive backup deployment.
- Shadow evaluation for parity checks without user impact.
- Gradual traffic shifting based on confidence metrics.

The stand-in server initiative in this internship adopts a similar backup-first approach, which enables risk-managed transition from vendor dependency to internal capability.

2.3 Event-Driven Fraud Data Synchronization

Fraud workflows demand low-latency data dissemination across many internal systems. Synchronous service chains can become fragile under scale and dependency failures. Event streaming platforms, such as Kafka, are therefore commonly adopted to decouple producers and consumers while maintaining high-throughput asynchronous processing.
In fraud scenarios, event-driven architectures offer:

- Near real-time propagation of fraud flags.
- Easy integration of additional consumers without changing producers.
- Better observability through topic lag and processing metrics.

At the same time, they require careful treatment of schema evolution, duplicate events, ordering guarantees, and idempotent consumption. These concerns are discussed later as part of limitations and future hardening.

2.4 Gap Addressed by This Internship Work

Based on the above context, the internship contributes by combining three complementary improvements in a single business domain:

- Automation and governance of decisioning artifact lifecycle.
- Cost and resilience strategy through an internal stand-in design.
- Real-time fraud-state centralization through event-driven integration.

This integrated perspective is valuable because business outcomes in banking are shaped by system behavior across the full lifecycle, not by isolated module-level optimizations.

3
Methodology

The internship execution followed an engineering methodology that combined problem decomposition, architecture-first planning, automation-driven implementation, and impact-oriented evaluation.

3.1 Workstream Decomposition

To keep delivery structured and measurable, the complete internship scope was divided into three workstreams:

1. Decisioning platform modernization and automation.
2. In-house stand-in decision server design.
3. Kafka-based fraud data centralization.

Each workstream had a dedicated problem statement, target outcome, and identified operational constraints.

3.2 Requirement Gathering and Validation

Requirements were interpreted from actual process behavior and stakeholder expectations:

- **Business teams:** faster and reliable policy/rule deployment.
- **Engineering teams:** reduced manual operations and maintainable architecture.
- **Operations and risk teams:** better traceability, resilience, and fraud response speed.

Validation was carried out through architecture reviews, implementation feasibility checks, and alignment with production-readiness principles.

3.3 Method for Workstream 1: Decisioning Automation

The approach for the first workstream involved:

- Identifying manual steps in the rule-to-artifact lifecycle.
- Replacing manual compilation with CI-triggered build execution.
- Standardizing artifact naming and versioning for `.adb` files.
- Publishing artifacts to Artifactory for controlled downstream consumption.

This method targeted release reliability and auditable deployment flow.

3.4 Method for Workstream 2: Stand-in Server Strategy

The stand-in initiative used a phased methodology:

- Define technical stack and service boundaries.
- Design APIs and integration points compatible with existing consumers.
- Plan fallback-first deployment instead of immediate full replacement.
- Use parity and confidence checkpoints before traffic expansion.

This methodology reduces migration risk and enables evidence-based adoption.

3.5 Method for Workstream 3: Fraud Centralization Pipeline

The fraud workflow method was event-driven:

- Publish incoming fraud-check events via Kafka producers.
- Consume events in central services through listeners.
- Persist blocked or under-investigation status in a central repository.
- Update the same record lifecycle after fraud review outcomes.

This method enables multi-team consistency and near real-time state propagation.

3.6 Evaluation Framework

The solutions were evaluated with practical enterprise criteria:

- **Speed:** reduction in manual processing time for rule updates.
- **Reliability:** consistency and repeatability of build and artifact release.
- **Scalability:** ability to support additional producers/consumers and services.
- **Governance:** traceability, version control, and operational audit readiness.
- **Cost impact:** long-term reduction in third-party dependency exposure.

3.7 Assumptions and Constraints

The following constraints influenced methodology decisions:

- Existing enterprise systems could not be abruptly replaced.
- Business approvals were necessary for traffic and policy-impacting changes.
- Security, compliance, and reliability requirements had to be preserved throughout.

Therefore, incremental rollout and controlled integration were prioritized over disruptive migrations.

4
Implementation

This chapter describes the technical implementation details of each internship workstream.

4.1 Workstream 1: Credit Decisioning Platform Modernization

4.1.1 Initial State

The initial platform had multiple utility repositories and a manual process for compiling rule updates into `.adb` artifacts. This model created delay and introduced avoidable human dependency in the release chain.

4.1.2 Repository Consolidation

As part of modernization, utility repositories were merged into a central codebase where feasible. The consolidation reduced code scattering, simplified dependency management, and improved discoverability of rule-related components.

4.1.3 Automated Rule Build Pipeline

A CI-triggered build workflow was introduced with the following logic:

1. Rule changes are committed and pushed.
2. Pipeline trigger starts automatic compilation.
3. Rules are compiled into deployable `.adb` artifacts.
4. Artifacts are validated and prepared for publishing.

This transformed a manual operational step into a repeatable machine-driven process.

4.1.4 Artifact Publishing and Versioning

Compiled artifacts are published to Artifactory with controlled versioning. Versioning provides:

- Historical traceability of deployed rule packages.
- Rollback capability in case of regression.
- Clear mapping between code revision and artifact revision.

4.2 Workstream 2: In-house Decision Stand-in Server

4.2.1 Architecture Selection

A prototype-ready architecture was selected:

- Backend: Java Spring Boot for robust service development and integration.
- Frontend: React + TypeScript for maintainable user-facing interfaces and internal operations tooling.

4.2.2 Operational Role

The stand-in server is designed as a backup-first component. It can be used when third-party decision providers are unavailable, delayed, or under maintenance. This design supports business continuity while preserving a low-risk path for incremental adoption.

4.2.3 Migration Strategy

Implementation planning included a phased migration strategy:

1. Enable passive readiness as contingency capability.
2. Execute parity testing with third-party decision outcomes.
3. Run controlled pilots in selected domains.
4. Gradually increase dependency on internal service as confidence grows.

4.3 Workstream 3: Fraud Central Repository with Kafka

4.3.1 Problem-Driven Design

Fraud alerts from identity theft workflows require rapid propagation across bank systems. To avoid inconsistent decisions across teams, a central repository model was designed using asynchronous event processing.

4.3.2 Event Flow Implementation

The implemented logical flow is:

1. A fraud-check transaction event is received by upstream systems.
2. Kafka producer publishes a standardized event to a topic.
3. Spring Boot Kafka listener consumes the event.
4. Consumer service writes/updates record in centralized repository.
5. Post-investigation status updates are consumed and persisted.

4.3.3 Core Design Considerations

To ensure production suitability, the implementation design includes:

- Idempotent event handling to limit duplicate-processing effects.
- Retry and dead-letter handling for transient failures.
- Status lifecycle updates to maintain auditable decision history.
- Monitoring hooks for consumer lag and processing health.

4.4 Technology Stack Summary

4.5 Implementation Limitations

Despite substantial progress, the following implementation limitations remain:

- End-to-end enterprise rollout requires cross-team production approvals.
- Stand-in service requires long-duration parity validation before critical migration.

No.	Technology	Purpose
1	Java Spring Boot	Backend services, integration APIs, Kafka listeners
2	React + TypeScript	Frontend interface for stand-in decision service
3	Kafka	Asynchronous event streaming for fraud workflows
4	Artifactory	Versioned storage of compiled `.adb` artifacts
5	CI Pipeline	Automated compile and publish lifecycle

Table 4.1: Technology stack used across internship workstreams.

- Fraud pipeline requires deeper hardening for exactly-once semantics and advanced governance.

5
Results and Discussion

This chapter summarizes the observed outcomes of the internship initiatives and discusses business relevance, technical implications, and identified limitations.

5.1 Results of Decisioning Modernization

The modernization and automation of decisioning artifacts provided clear operational improvements:

- Rule changes became pipeline-driven instead of manually compiled.
- Artifact publication became standardized through Artifactory.
- Versioning improved traceability and rollback readiness.
- Consolidation reduced complexity caused by fragmented repositories.

Discussion: The primary gain is not only speed but reliability under repeated releases. In regulated financial systems, consistent and auditable release behavior is as critical as low deployment latency.

5.2 Results of Stand-in Server Initiative

The stand-in server initiative established a practical path toward reducing external dependency.

- Internal architecture for fallback decisioning was defined.
- Technology stack decisions were aligned with enterprise compatibility.
- Phased migration strategy reduced operational risk.

Discussion: The largest value is strategic. Even before full replacement, a reliable backup service improves continuity and strengthens operational resilience during third-party outages.

5.3 Results of Fraud Centralization Pipeline

The event-driven fraud architecture produced a structured mechanism for cross-team visibility.

- Incoming fraud-check events can be streamed through Kafka.
- Consumers update a centralized repository for blocked and investigation states.
- Post-investigation updates close the lifecycle loop for consistent status tracking.

Discussion: A centralized fraud state reduces risk of contradictory decisions across banking units. This is particularly important in identity theft cases where delayed synchronization can cause severe compliance and financial impact.

5.4 Measurable Outcomes Summary

Initiative	**KPI**	**Baseline Challenge**	**Post-Implementation Outcome**	**Business Impact**
Decisioning modernization	Rule release lead time; manual steps per release; artifact traceability rate	Manual rule compile and publish process with fragmented ownership	Push-triggered automated compile and publish path established; versioned `.adb` artifacts stored in Artifactory	Faster policy rollout and stronger audit readiness
Stand-in decision server	Internal decision coverage (%); third-party dependency ratio; continuity readiness score	High external dependency for cloud/local third-party engines and associated recurring cost	In-house backup architecture defined (Spring Boot + React/TS) with phased adoption strategy and parity-first rollout model	Improved resilience posture and long-term cost optimization path
Fraud centralization pipeline	Fraud state propagation latency; cross-team consistency rate; stale-status incidence	No unified repository for blocked/under-investigation transactions	Kafka producer-consumer flow with central repository updates and lifecycle status synchronization implemented	Faster fraud response and reduced contradictory decision risk

Table 5.1: Measurable outcomes framework across internship initiatives.

Recommended KPI tracking formulas for deployment dashboards:

- **Rule Release Lead Time** = *Artifact Published Timestamp - Rule Commit Timestamp*
- **Internal Decision Coverage (%)** $= \frac{\text{Requests served by stand-in}}{\text{Total decision requests}} \times 100$
- **Fraud Propagation Latency** = *Central Repository Update Timestamp - Fraud Event Publish Timestamp*
- **Cross-Team Consistency Rate (%)** $= \frac{\text{Decisions aligned with latest fraud status}}{\text{Total evaluated decisions}} \times 100$

5.5 Limitations and Risks

Although the outcomes are positive, some limitations remain:

- Pipeline reliability must be maintained to avoid build bottlenecks.
- Stand-in service requires sustained parity verification against existing engines.
- Kafka-based systems require disciplined handling of ordering, retries, and duplicates.
- Data governance is critical for sensitive fraud records and lifecycle retention.

5.6 Key Learnings from Discussion

The discussion across all workstreams highlights a common pattern: enterprise engineering success depends on combining automation, architecture discipline, and controlled rollout. High-quality software outcomes in banking are created when technical decisions are tied directly to business risk and compliance needs.

6
Conclusions and Future Works

This internship at Wells Fargo provided practical exposure to enterprise-scale engineering where reliability, governance, and response speed are as important as feature delivery. Through the three workstreams, I contributed to decisioning modernization, cost-aware architecture strategy, and fraud data centralization.

6.1 Conclusions

The major conclusions are as follows:

- Automating rule compilation and artifact publication substantially improves consistency and reduces manual operational risk.
- Repository consolidation improves maintainability and lowers cognitive overhead for development teams.
- In-house stand-in decisioning capability is a viable strategic direction for reducing long-term vendor dependency and improving service continuity.
- Event-driven fraud centralization is essential for maintaining a unified risk view across teams and preventing contradictory decisions.

Overall, the internship demonstrated that sustainable impact comes from systems thinking: aligning architecture choices with business goals, compliance constraints, and operational realities.

6.2 Future Works

Future scope for this work includes:

1. **Decisioning automation hardening**

- Add rule schema validation and policy quality gates before compilation.
- Introduce approval-driven promotion flow across environments.

2. **Stand-in server productionization**
   - Build parity dashboards comparing internal and third-party outcomes.
   - Execute phased traffic migration with rollback guardrails.
3. **Fraud pipeline resilience**
   - Strengthen idempotency controls and event contract governance.
   - Add dead-letter queue analytics and lag-based alerting.

6.3 Personal Learning Outcomes

6.3.1 Technical and Professional Learning

I learned to translate business pain points into implementable technical architecture, design migration paths with controlled risk, and evaluate solutions through enterprise non-functional requirements such as reliability, traceability, and scalability.

6.3.2 Learning to Use GitHub Copilot Better

During this internship, I improved how I use GitHub Copilot in practical engineering tasks:

- I learned to write context-rich prompts so generated code better matches domain requirements.
- I used Copilot to accelerate boilerplate, refactoring, and documentation drafting while keeping validation responsibility with myself.
- I used iterative prompting to compare alternatives for readability, maintainability, and production safety.
- I learned that Copilot is most effective as a productivity partner when paired with strong engineering judgment and testing discipline.

These learnings improved both development speed and communication quality during project execution.

Bibliography

[1] Spring Team, *Spring Boot Reference Documentation*, VMware, 2025.

[2] Apache Software Foundation, *Apache Kafka Documentation*, Version 3.x Documentation Set, 2025.

[3] Meta Open Source, *React Documentation*, Official React Docs, 2025.

[4] Microsoft, *TypeScript Handbook*, Official TypeScript Documentation, 2025.

[5] Jez Humble and David Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.

[6] Ben Stopford, *Designing Event-Driven Systems*, O'Reilly Media, 2018.